

SIH-2026 Form Fill Sheet

Copy these answers into the Google Form on your phone. Do not write Flask, PostgreSQL or CNN.

Form: <https://docs.google.com/forms/d/1joANngfzID5rSXp9-eraprx-JfbYyuRRnzOVC7dNMM/viewform>

1. Name of Student

HRITIK KUMAR (select from dropdown)

2–4. Fill yourself

Whatsapp no: _____

Email Id: _____

Branch: CSE / CS_AI / CS_AIML / CS_DS / IT / ECE / MCA / MBA

Section: A–R or Other

Year: II / III / IV

5. Write Problem statement

Phishing kits impersonate trusted domains and inject social-engineering payloads into SMS, WhatsApp and email. A URL-only checker fails when the attack is in-page or in-message. PhishShield AI is a real-time browser protection layer (Chrome Manifest V3) backed by a FastAPI threat engine. The extension intercepts the active tab, scores the URL and visible links against the same pipeline as the dashboard, and raises HIGH/CRITICAL overlays before credential entry.

6. Objective of the Problem Statement (5 points)

1. Provide real-time in-browser protection so a phishing page is scored before the user enters a password, OTP or card detail.
2. Make the Chrome Manifest V3 extension the main defence layer: auto-scan the active tab, badge the risk (OK / MED / HIGH / CRIT), and block HIGH/CRITICAL sites with an overlay.
3. Detect impersonation not only from the address bar but also from visible page links, shortened URLs and QR-borne URLs, using one threat API.

4. Produce an explainable 0–100 risk score (heuristics + intel + psychological markers), with Simple View for users and Technical View for audit.
5. Keep dashboard and extension on the same FastAPI engine, so a URL scanned in Chrome and in the web app returns the same verdict.

7. Used Technology (min. 2 points)

1. Chrome Extension — Manifest V3 (service worker, content script, popup, chrome.storage)
2. FastAPI + Uvicorn + Pydantic (REST threat engine)
3. React 19 + Vite 8 (operator dashboard)
4. SQLite (threats.db) for scans, users, reputation
5. JWT (HS256) + bcrypt; OpenCV QR decode; optional Google Safe Browsing / OpenPhish

Do not write: Flask, live PostgreSQL, TensorFlow/Keras CNN, sklearn model.

8. Methodology (10 points)

1. Extension service worker reads the active-tab URL; skip chrome:// and edge:// pages.
2. Content script collects visible http(s) links only (no passwords, OTP, cookies, SMS).
3. Popup / auto-scan POSTs to FastAPI (/api/v1/threat/check-url or /api/v1/analyze/content).
4. URLChecker runs TLD, allowlist, brand-typo, shortener expansion and keyword heuristics.
5. Fail-soft intel: DNS + RDAP age; Google Safe Browsing if keyed, else OpenPhish feed.
6. Message path: substring psychology detector (urgency, fear, OTP/KYC, countdown).
7. Fusion engine: weighted signals (url 0.22, domain 0.16, intel 0.10, psych 0.10, ...); psychology-only score capped at 35.

8. Persist url_checks / content_scans in SQLite; toolbar badge = OK / MED / HIGH / CRIT.
9. HIGH/CRITICAL → full-page overlay (Go back / Continue anyway); toast for other levels.
10. Dashboard consumes the same JSON for history, reports and Simple/Technical View.